

GOOGLE CLOUD PROFESSIONAL CLOUD ARCHITECT

Section 4.2 — Managing Google Kubernetes Engine Resources

GCP Associate Cloud Engineer

Study Notes Study Notes April 2026

Exam Relevance

This topic is part of **Section 4: Ensuring successful operation of a cloud solution (~20 % of the exam)**. You must know how to view cluster inventory, configure access to Artifact Registry, work with node pools, manage Kubernetes resources, and configure autoscaling.

1. Viewing Current Running Cluster Inventory

Cluster-Level Commands

```
# List all GKE clusters in the project
gcloud container clusters list

# Describe a specific cluster
gcloud container clusters describe my-cluster --zone=us-central1-a

# Get cluster credentials (connect kubectl)
gcloud container clusters get-credentials my-cluster --zone=us-central1-a
```

Node Inventory

```
# List all nodes in the cluster
kubectl get nodes

# List nodes with detailed info (IPs, status, roles, version)
kubectl get nodes -o wide

# Describe a specific node (capacity, conditions, pods running on it)
kubectl describe node NODE_NAME

# List nodes with labels
kubectl get nodes --show-labels

# List nodes with specific label
kubectl get nodes -l cloud.google.com/gke-nodepool=default-pool

# Get node resource usage
kubectl top nodes
```

Pod Inventory

```
# List all pods in the default namespace
kubectl get pods

# List pods in all namespaces
kubectl get pods --all-namespaces
kubectl get pods -A

# List pods with detailed info
kubectl get pods -o wide

# List pods in a specific namespace
kubectl get pods -n my-namespace

# Describe a pod (events, conditions, containers)
kubectl describe pod POD_NAME

# Get pod logs
kubectl logs POD_NAME
kubectl logs POD_NAME -c CONTAINER_NAME # Multi-container pod
kubectl logs POD_NAME --previous        # Previous crashed container
kubectl logs POD_NAME -f                 # Stream logs

# Get pod resource usage
kubectl top pods

# List pods with labels
kubectl get pods --show-labels
kubectl get pods -l app=my-app
```

Service Inventory

```
# List all services
kubectl get services
kubectl get svc

# List services in all namespaces
kubectl get svc -A

# Describe a service
kubectl describe svc my-service

# Get service endpoints
kubectl get endpoints my-service
```

2. Configuring GKE to Access Artifact Registry

What Is Artifact Registry?

- Google's managed **container image and package repository**
- Successor to Container Registry (gcr.io)
- Supports Docker, Maven, npm, Python, Go, and more

Creating an Artifact Registry Repository

```
# Create a Docker repository
gcloud artifacts repositories create my-repo \
  --repository-format=docker \
  --location=us-central1 \
  --description="My Docker images"

# List repositories
gcloud artifacts repositories list

# Configure Docker authentication to Artifact Registry
gcloud auth configure-docker us-central1-docker.pkg.dev
```

Image URL Format

```
LOCATION-docker.pkg.dev/PROJECT_ID/REPOSITORY/IMAGE:TAG

Example:
us-central1-docker.pkg.dev/my-project/my-repo/my-app:v1
```

GKE Access to Artifact Registry

Method 1: Default (Node Service Account)

GKE nodes use their service account to pull images. The service account needs the **Artifact Registry Reader** role:

```
# Grant Artifact Registry Reader to the node service account
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:NODE_SA@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/artifactregistry.reader"
```

Method 2: Workload Identity (Recommended)

Map a Kubernetes service account to a Google service account with Artifact Registry access:

```
# Grant Artifact Registry Reader to the GCP service account
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:my-gcp-sa@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/artifactregistry.reader"

# Set up Workload Identity binding
gcloud iam service-accounts add-iam-policy-binding \
  my-gcp-sa@PROJECT_ID.iam.gserviceaccount.com \
  --role="roles/iam.workloadIdentityUser" \
  --member="serviceAccount:PROJECT_ID.svc.id.goog[NAMESPACE/KSA_NAME] "
```

Method 3: ImagePullSecrets

For cross-project or external registries:

```
# Create a secret for pulling images
kubectl create secret docker-registry my-registry-key \
  --docker-server=us-central1-docker.pkg.dev \
  --docker-username=_json_key \
  --docker-password="$(cat key.json)" \
  --docker-email=sa@project.iam.gserviceaccount.com
```

```
# Reference in pod spec
spec:
  imagePullSecrets:
    - name: my-registry-key
  containers:
    - name: my-app
      image: us-central1-docker.pkg.dev/PROJECT/REPO/IMAGE:TAG
```

Pushing Images to Artifact Registry

```
# Build and push with Docker
docker build -t us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1 .
docker push us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1

# Build and push with Cloud Build
gcloud builds submit --tag us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1
```

3. Working with Node Pools

What Are Node Pools?

- A **group of nodes within a cluster** that share the same configuration
- A cluster can have **multiple node pools** with different configurations
- Each node pool specifies: machine type, disk size, number of nodes, labels, taints

Adding a Node Pool

```
# Add a new node pool
gcloud container node-pools create my-new-pool \
  --cluster=my-cluster \
  --zone=us-central1-a \
  --machine-type=n2-standard-8 \
  --num-nodes=3 \
  --disk-size=100GB \
  --disk-type=pd-ssd

# Add a node pool with autoscaling
gcloud container node-pools create autoscale-pool \
  --cluster=my-cluster \
  --zone=us-central1-a \
  --machine-type=e2-standard-4 \
  --enable-autoscaling \
  --min-nodes=1 \
  --max-nodes=10

# Add a node pool with Spot VMs
gcloud container node-pools create spot-pool \
  --cluster=my-cluster \
  --zone=us-central1-a \
  --machine-type=e2-standard-4 \
  --spot \
  --num-nodes=5

# Add a node pool with GPUs
gcloud container node-pools create gpu-pool \
  --cluster=my-cluster \
  --zone=us-central1-a \
  --machine-type=n1-standard-4 \
  --accelerator=type=nvidia-tesla-t4,count=1 \
  --num-nodes=2

# Add a node pool with labels and taints
gcloud container node-pools create labeled-pool \
  --cluster=my-cluster \
  --zone=us-central1-a \
  --machine-type=e2-standard-4 \
  --num-nodes=3 \
  --node-labels=workload=batch,tier=backend \
  --node-taints=dedicated=batch:NoSchedule
```

Editing a Node Pool

```
# Enable autoscaling on an existing pool
gcloud container node-pools update my-pool \
  --cluster=my-cluster \
  --zone=us-central1-a \
  --enable-autoscaling \
  --min-nodes=2 \
  --max-nodes=10

# Resize a node pool (manual scaling)
gcloud container clusters resize my-cluster \
  --node-pool=my-pool \
  --num-nodes=5 \
  --zone=us-central1-a

# Enable auto-repair
gcloud container node-pools update my-pool \
  --cluster=my-cluster \
  --zone=us-central1-a \
  --enable-autorepair

# Enable auto-upgrade
gcloud container node-pools update my-pool \
  --cluster=my-cluster \
  --zone=us-central1-a \
  --enable-autoupgrade

# Update node pool labels
gcloud container node-pools update my-pool \
  --cluster=my-cluster \
  --zone=us-central1-a \
  --node-labels=environment=production
```


Removing a Node Pool

```
# Delete a node pool
gcloud container node-pools delete my-pool \
  --cluster=my-cluster \
  --zone=us-central1-a

# List node pools
gcloud container node-pools list \
  --cluster=my-cluster \
  --zone=us-central1-a

# Describe a node pool
gcloud container node-pools describe my-pool \
  --cluster=my-cluster \
  --zone=us-central1-a
```

4. Working with Kubernetes Resources

Pods

```
# Create a pod from a YAML file
kubectl apply -f pod.yaml

# Run a one-off pod
kubectl run my-pod --image=nginx --restart=Never

# Run a pod and open a shell
kubectl run debug --image=busybox -it --rm -- /bin/sh

# Delete a pod
kubectl delete pod my-pod

# Force delete a stuck pod
kubectl delete pod my-pod --grace-period=0 --force

# Copy files to/from a pod
kubectl cp local-file.txt POD_NAME:/path/in/pod
kubectl cp POD_NAME:/path/in/pod/file.txt ./local-file.txt
```

Services

```
# ClusterIP Service (internal only)
apiVersion: v1
kind: Service
metadata:
  name: my-internal-service
spec:
  type: ClusterIP
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080

---
# LoadBalancer Service (external)
apiVersion: v1
kind: Service
metadata:
  name: my-external-service
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080

---
# NodePort Service
apiVersion: v1
kind: Service
metadata:
  name: my-nodeport-service
spec:
  type: NodePort
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080
```

StatefulSets

StatefulSets manage **stateful applications** (databases, message queues) with: - **Stable network identities** (pod-0, pod-1, pod-2) - **Stable persistent storage** (each pod gets its own

PVC) - **Ordered deployment and scaling** (pods created in order: 0, 1, 2) - **Ordered rolling updates**

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 10Gi
```

```
# List StatefulSets
kubectl get statefulsets
kubectl get sts

# Scale a StatefulSet
kubectl scale statefulset postgres --replicas=5

# Delete a StatefulSet (pods are deleted in reverse order)
kubectl delete statefulset postgres
```

Deployments vs. StatefulSets

Feature	Deployment	StatefulSet
Pod identity	Random names	Ordered (pod-0, pod-1)
Storage	Shared PVC (if any)	Individual PVCs per pod
Scaling	Parallel	Ordered
Updates	Rolling (parallel)	Rolling (ordered)
Use case	Stateless apps	Stateful apps (databases)

5. Managing Autoscaling Configurations

Horizontal Pod Autoscaler (HPA)

Scales the **number of pod replicas** based on metrics:

```
# Create HPA based on CPU
kubectl autoscale deployment my-app \
  --min=2 \
  --max=10 \
  --cpu-percent=70

# Create HPA from YAML
```

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
```

```
# View HPA status
kubectl get hpa
kubectl describe hpa my-app-hpa

# Delete HPA
kubectl delete hpa my-app-hpa
```

Vertical Pod Autoscaler (VPA)

Adjusts **CPU and memory requests** for pods:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: my-app-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  updatePolicy:
    updateMode: "Auto" # Auto, Recreate, Initial, Off
  resourcePolicy:
    containerPolicies:
      - containerName: my-app
        minAllowed:
          cpu: "100m"
          memory: "128Mi"
        maxAllowed:
          cpu: "4"
          memory: "8Gi"
```

VPA Update Modes

Mode	Description
Off	Only provides recommendations (no action)
Initial	Sets resources only at pod creation
Recreate	Evicts and recreates pods with new resources
Auto	Similar to Recreate (applies in-place when available)

Cluster Autoscaler

Scales the **number of nodes** in a node pool:

```
# Enable cluster autoscaler
gcloud container clusters update my-cluster \
  --zone=us-central1-a \
  --enable-autoscaling \
  --min-nodes=2 \
  --max-nodes=10 \
  --node-pool=default-pool

# Disable autoscaler
gcloud container clusters update my-cluster \
  --zone=us-central1-a \
  --no-enable-autoscaling \
  --node-pool=default-pool
```

HPA vs. VPA vs. Cluster Autoscaler

Autoscaler	What It Scales	Based On
HPA	Number of pod replicas	CPU, memory, custom metrics
VPA	CPU/memory requests per pod	Historical resource usage
Cluster Autoscaler	Number of nodes	Pending pods (unschedulable)

How They Work Together

```
Traffic increase
→ HPA creates more pods
→ Pods become unschedulable (not enough node capacity)
→ Cluster Autoscaler adds more nodes
→ Pods get scheduled on new nodes

Traffic decrease
→ HPA reduces pod count
→ Nodes become underutilized
→ Cluster Autoscaler removes nodes (after cool-down)
```

8. Cluster Upgrades

- GKE upgrades: control plane first, then node pools

- Release channels: Rapid, Regular, Stable, Extended — GKE auto-upgrades within the channel
- Manual upgrade:

```
# Upgrade control plane to specific version
gcloud container clusters upgrade MY_CLUSTER --master --cluster-version=1.29.0 --zone=ZONE
# Upgrade a node pool
gcloud container clusters upgrade MY_CLUSTER --node-pool=MY_POOL --cluster-version=1.29.0 --zone=ZONE
```

- Surge upgrade settings (control disruption):

```
gcloud container node-pools update MY_POOL --cluster=MY_CLUSTER --zone=ZONE \
  --max-surge-upgrade=1 --max-unavailable-upgrade=0
```

- Maintenance windows:

```
gcloud container clusters update MY_CLUSTER --zone=ZONE \
  --maintenance-window-start="2024-01-01T09:00:00Z" \
  --maintenance-window-end="2024-01-01T17:00:00Z" \
  --maintenance-window-recurrence="FREQ=WEEKLY;BYDAY=SA,SU"
```

- Exam tip: Control plane and node pool versions can differ by at most 2 minor versions; upgrade control plane before nodes

9. ConfigMaps and Secrets

ConfigMaps:

```
# Create from literal values
kubectl create configmap my-config --from-literal=DB_HOST=localhost --from-literal=DB_PORT=5432
# Create from file
kubectl create configmap my-config --from-file=config.properties
# View
kubectl get configmap my-config -o yaml
```

Use in a Pod:


```
spec:
  containers:
  - name: app
    envFrom:
    - configMapRef:
        name: my-config
    # OR as a volume
    volumeMounts:
    - name: config-vol
      mountPath: /etc/config
  volumes:
  - name: config-vol
    configMap:
      name: my-config
```

Secrets:

```
kubectl create secret generic my-secret --from-literal=password=s3cr3t
kubectl create secret docker-registry gcr-secret \
  --docker-server=us-docker.pkg.dev \
  --docker-username=_json_key \
  --docker-password="$(cat key.json)"
kubectl get secret my-secret -o jsonpath='{.data.password}' | base64 --decode
```

Use Secret Manager instead of K8s Secrets for sensitive data — integrate via the Secret Manager add-on or mount with Workload Identity.

10. Network Policies

- Controls pod-to-pod and pod-to-external traffic (requires `--enable-network-policy` on cluster)
- Default: all pods can communicate with all pods
- NetworkPolicy resources are namespaced

Example — allow only pods with label `app=frontend` to reach `app=backend` on port 8080:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 8080
```

- Exam tip: An empty `podSelector` matches ALL pods in the namespace; an empty `ingress` list means DENY ALL ingress

Exam Practice Questions

1. A pod is stuck in “Pending” state. `kubectl describe` shows “Insufficient cpu”. What’s happening and how do you fix it?
 - Answer: The cluster doesn’t have enough CPU capacity. Either **resize the node pool** manually or **enable the cluster autoscaler** to automatically add nodes.
2. You need to deploy a PostgreSQL database in GKE with persistent storage that survives pod restarts. Which resource should you use?
 - Answer: Use a **StatefulSet** with `volumeClaimTemplates`. Each pod gets a dedicated PersistentVolumeClaim that persists across pod restarts.
3. Your GKE pods need to pull images from an Artifact Registry in a different project. What’s the recommended approach?
 - Answer: Use **Workload Identity** to map the pod’s Kubernetes service account to a Google Cloud service account that has `roles/artifactregistry.reader` in the target project.
4. You want to automatically adjust pod CPU requests based on actual usage to right-size resources. Which autoscaler should you use?

- Answer: **Vertical Pod Autoscaler (VPA)** — It monitors actual resource usage and adjusts CPU/memory requests accordingly.

5. **How can you add GPU-capable nodes to an existing GKE cluster without affecting current workloads?**

- Answer: Create a **new node pool** with GPU-capable machine types and `--accelerator` flag. Use **node taints** to ensure only GPU workloads are scheduled there.

6. **You need to view all pods running across all namespaces, including system pods. What command do you use?**

- Answer: `kubect1 get pods --all-namespaces` or `kubect1 get pods -A`.

Glossary

Artifact Registry — GCP's managed repository service for storing and managing container images and language packages (Docker, Maven, npm, Python, Go); the successor to Container Registry (gcr.io).

Artifact Registry Reader — A GCP IAM role (`roles/artifactregistry.reader`) that grants read-only access to pull container images from an Artifact Registry repository.

Auto-repair — A GKE node pool feature that automatically detects and replaces unhealthy nodes, restoring them to a healthy running state without manual intervention.

Auto-upgrade — A GKE node pool feature that automatically upgrades nodes to newer Kubernetes versions within the configured release channel, reducing manual maintenance.

Autoscaling — The automatic adjustment of the number of pods or nodes in a GKE cluster in response to workload demand; implemented via HPA, VPA, or the Cluster Autoscaler.

Base64 — A binary-to-text encoding scheme used to encode Kubernetes Secret data; Secret values are stored as base64-encoded strings and must be decoded when retrieved.

Busybox — A minimal Linux container image commonly used for debugging and troubleshooting in Kubernetes environments.

Cluster Autoscaler — A GKE component that automatically adjusts the number of nodes in a node pool based on whether pods are pending (unschedulable) due to insufficient resources.

ClusterIP — A Kubernetes Service type that exposes the service on an internal cluster-only IP address; the default service type, accessible only within the cluster.

ConfigMap — A Kubernetes resource that stores non-sensitive configuration data as key-value pairs, which can be injected into pods as environment variables or mounted as files.

Container Registry (gcr.io) — The predecessor to Artifact Registry for storing Docker container images on GCP; being replaced by Artifact Registry.

Control Plane — The set of components in a Kubernetes cluster (API server, etcd, scheduler, controller manager) that manage the cluster state; in GKE, the control plane is Google-managed.

CPU Request — The amount of CPU a Kubernetes container is guaranteed to receive; used by the scheduler to decide which node to place a pod on.

Deployment — A Kubernetes resource that manages a set of identical, stateless pod replicas with support for rolling updates, rollbacks, and scaling.

Docker — A platform for building, shipping, and running containers; GKE nodes pull Docker-formatted container images from registries like Artifact Registry.

etcd — A distributed key-value store used by Kubernetes as its primary data store for all cluster state and configuration; managed by Google in GKE.

GCP (Google Cloud Platform) — Google's suite of cloud computing services; GKE is one of its core managed services.

GKE (Google Kubernetes Engine) — GCP's fully managed Kubernetes service that automates the deployment, scaling, and management of containerized applications using Kubernetes.

GPU (Graphics Processing Unit) — A specialized processor used for machine learning and compute-intensive workloads; GKE supports GPU node pools via the `--accelerator` flag.

HA (High Availability) — A design principle that ensures a system remains operational with minimal downtime; achieved in GKE through multi-zone node pools and regional clusters.

HPA (Horizontal Pod Autoscaler) — A Kubernetes resource that automatically scales the number of pod replicas in a Deployment or StatefulSet based on CPU, memory, or custom metrics.

IAM (Identity and Access Management) — GCP's access control system for managing who has what permissions on which resources; used to grant Artifact Registry and Workload Identity permissions.

ImagePullSecrets — Kubernetes Secret objects that store credentials for authenticating to private container registries; referenced in pod specs to allow image pulls.

Ingress (NetworkPolicy) — In the context of Kubernetes NetworkPolicy, traffic flowing into a pod from other pods or external sources; controlled using `policyTypes: Ingress` rules.

KSA (Kubernetes Service Account) — A Kubernetes identity used by pods to authenticate to the Kubernetes API and, via Workload Identity, to GCP APIs.

kubectl — The command-line interface for managing Kubernetes clusters; used to create, view, and delete resources such as pods, services, deployments, and ConfigMaps.

Kubernetes — An open-source container orchestration system for automating deployment, scaling, and management of containerized applications; the foundation of GKE.

LoadBalancer — A Kubernetes Service type that provisions an external GCP load balancer and assigns a public IP address to expose the service to internet traffic.

Maintenance Window — A configurable time period during which GKE is allowed to perform auto-upgrades and maintenance operations on clusters and node pools.

Memory Request — The amount of memory a Kubernetes container is guaranteed to receive; used by the scheduler to determine node placement for pods.

Namespace — A Kubernetes mechanism for isolating resources within a cluster; used to separate workloads by team, environment, or application.

Network Policy — A Kubernetes resource that controls how pods communicate with each other and with external endpoints; requires the `--enable-network-policy` flag on GKE clusters.

Node — A worker machine (Compute Engine VM) in a GKE cluster that runs containerized workloads; managed by the control plane.

Node Pool — A group of nodes within a GKE cluster that share the same configuration (machine type, disk size, labels, taints); a cluster can have multiple node pools.

NodePort — A Kubernetes Service type that exposes the service on a static port on each cluster node's IP address, enabling external access without a cloud load balancer.

NVIDIA Tesla T4 — A GPU model from NVIDIA supported in GKE GPU node pools for machine learning and compute workloads.

PersistentVolumeClaim (PVC) — A Kubernetes resource that requests storage from a StorageClass; used by StatefulSets to provide each pod with its own persistent storage.

Pod — The smallest deployable unit in Kubernetes; a group of one or more containers that share network and storage resources and run on the same node.

PodSelector — A label-based selector in Kubernetes NetworkPolicy that identifies which pods the policy applies to; an empty podSelector matches all pods in the namespace.

PostgreSQL — An open-source relational database; used as a StatefulSet example in GKE deployments, with each replica requiring its own PersistentVolumeClaim.

Release Channel — A GKE setting (Rapid, Regular, Stable, Extended) that determines which Kubernetes version a cluster receives and the cadence of automatic upgrades.

Replica — A copy of a pod managed by a Deployment or StatefulSet; the number of replicas determines how many pod instances run simultaneously.

Secret — A Kubernetes resource that stores sensitive data (passwords, tokens, keys) as base64-encoded values; should be replaced with Secret Manager for production workloads.

Secret Manager — A GCP service for securely storing and managing sensitive configuration data (API keys, passwords, certificates); recommended over Kubernetes Secrets.

Secret Manager Add-on — A GKE add-on that allows Secret Manager secrets to be automatically synced and mounted into pods via the Secrets Store CSI Driver.

Service — A Kubernetes resource that provides a stable network endpoint (IP address and DNS name) for a set of pods selected by labels.

Spot VM — A Compute Engine VM type used in GKE node pools that offers lower cost by using spare capacity but can be preempted by GCP with short notice.

StatefulSet — A Kubernetes workload resource for managing stateful applications (databases, message queues) that require stable network identities, persistent storage, and ordered deployment.

Surge Upgrade — A GKE node pool upgrade strategy that creates additional (surge) nodes before removing old ones, controlled by `--max-surge-upgrade` and `--max-unavailable-upgrade` settings.

Taint — A Kubernetes node property that repels pods from being scheduled on a node unless the pod has a matching toleration; used to dedicate nodes to specific workloads (e.g., `dedicated=batch:NoSchedule`).

TCP (Transmission Control Protocol) — A connection-oriented transport protocol; used in Kubernetes NetworkPolicy and Service port definitions.

VPA (Vertical Pod Autoscaler) — A Kubernetes resource that automatically adjusts the CPU and memory requests of pods based on historical resource usage, helping right-size pod allocations.

Workload Identity — A GKE feature that allows Kubernetes service accounts (KSAs) to impersonate GCP service accounts (GSAs), enabling pods to authenticate to GCP APIs without storing keys.

Zone — A geographically isolated deployment area within a GCP region; GKE node pools can span a single zone or multiple zones for high availability.